



ASIGNATURA DOSW	TECNOLOGÍA Java 21 o 24	EJERCICIOS 16	NOMBRE / FECHA _____
---------------------------	-----------------------------------	-------------------------	-------------------------

Instrucciones: Para cada ejercicio, lea el enunciado, analice los datos de entrada y escriba el código Java usando Streams y expresiones Lambda que produzca la salida esperada.

#01**Filtrar por letra inicial**`filter()`**ENUNCIADO**

Dada una lista de nombres de estudiantes, obtenga una nueva lista que contenga únicamente los nombres que empiezan por la letra A.

DATOS DE ENTRADA	SALIDA ESPERADA
<code>["Ana", "Carlos", "Andres", "Pedro", "Alejandra", "Juan", "Amanda"]</code>	<code>["Ana", "Andres", "Alejandra", "Amanda"]</code>

#02**Imprimir elementos de una colección**`forEach()`**ENUNCIADO**

Dada una lista de productos, recorra la colección e imprima cada elemento en consola con el mensaje: "Producto disponible: <nombre>"

DATOS DE ENTRADA	SALIDA ESPERADA
<code>["Laptop", "Mouse", "Teclado", "Monitor", "Impresora"]</code>	<code>Producto disponible: Laptop Producto disponible: Mouse Producto disponible: Teclado Producto disponible: Monitor Producto disponible: Impresora</code>

#03**Transformar a mayúsculas**`map()`**ENUNCIADO**

Dada una lista de nombres de ciudades, genere una nueva lista donde todas estén escritas completamente en mayúsculas.

DATOS DE ENTRADA	SALIDA ESPERADA
<code>["Bogotá", "Medellín", "Cali", "Barranquilla"]</code>	<code>["BOGOTÁ", "MEDELLÍN", "CALI", "BARRANQUILLA"]</code>

#04

Suma total de una lista

`reduce()`

ENUNCIADO

Dada una lista de números enteros, calcule la suma total de todos los elementos utilizando una operación de reducción.

DATOS DE ENTRADA	SALIDA ESPERADA
<code>[12, 8, 5, 10, 15]</code>	<code>suma = 50</code>

#05

Eliminar duplicados con colector

`collect()`

ENUNCIADO

Dada una lista de correos electrónicos donde algunos están repetidos, transforme la colección en una estructura que elimine automáticamente los elementos duplicados.

DATOS DE ENTRADA	SALIDA ESPERADA
<code>["a@correo.com", "b@correo.com", "a@correo.com", "c@correo.com", "b@correo.com"]</code>	<code>Set con 3 elementos únicos: "a@correo.com" "b@correo.com" "c@correo.com"</code>

#06

Registrar transformaciones intermedias

`peek()`

ENUNCIADO

Dada una lista de empleados, convierta los nombres a mayúsculas y registre en consola cada transformación realizada antes de guardar el resultado final en una lista.

DATOS DE ENTRADA	SALIDA ESPERADA
------------------	-----------------

```
["Laura", "Pedro", "Carlos", "Ana"]
```

```
Transformado: LAURA  
Transformado: PEDRO  
Transformado: CARLOS  
Transformado: ANA  
Lista: ["LAURA", "PEDRO", "CARLOS", "ANA"]
```

#07

Ordenar una colección

`sorted()`

ENUNCIADO

Dada una lista de edades, obtenga una nueva lista ordenada de menor a mayor y otra de mayor a menor.

DATOS DE ENTRADA

```
[25, 18, 32, 21, 19, 28]
```

SALIDA ESPERADA

```
Ascendente:  
[18, 19, 21, 25, 28, 32]
```

```
Descendente:  
[32, 28, 25, 21, 19, 18]
```

#08

Eliminar valores repetidos

`distinct()`

ENUNCIADO

Dada una lista de códigos de producto con repeticiones, genere una nueva colección donde cada código aparezca una sola vez.

DATOS DE ENTRADA

```
["P01", "P02", "P01",  
 "P03", "P02", "P04"]
```

SALIDA ESPERADA

```
["P01", "P02", "P03", "P04"]
```

#09

Limitar cantidad de resultados

`limit()`

ENUNCIADO

Dada una lista de los 20 mejores puntajes de un videojuego, obtenga únicamente los primeros 5 elementos para mostrar el ranking principal.

DATOS DE ENTRADA	SALIDA ESPERADA
<pre>Lista de 20 puntajes ordenados de mayor a menor.</pre>	<pre>Lista con los 5 primeros puntajes del ranking (top 5 únicamente).</pre>

#10

Saltar los primeros elementos

`skip()`

ENUNCIADO

Dada una lista de películas, ignore las dos primeras y obtenga una nueva lista con las restantes.

DATOS DE ENTRADA	SALIDA ESPERADA
<pre>["Avatar", "Titanic", "Interestelar", "Matrix", "Gladiador"]</pre>	<pre>["Interestelar", "Matrix", "Gladiador"]</pre>

#11

Encontrar el valor mínimo

`min(Comparator)`

ENUNCIADO

Dada una lista de precios, determine cuál es el precio más bajo utilizando Streams.

DATOS DE ENTRADA	SALIDA ESPERADA
<pre>[12000, 5000, 18000, 7500, 3000]</pre>	<pre>Precio mínimo: 3000</pre>

#12

Encontrar el valor máximo

`max(Comparator)`

ENUNCIADO

Dada una lista de salarios, determine cuál es el salario más alto utilizando Streams.

DATOS DE ENTRADA	SALIDA ESPERADA
<pre>[1800000, 2500000, 3200000, 2100000, 4000000]</pre>	<pre>Salario máximo: 4000000</pre>

#13

Verificar si existe al menos un par

`anyMatch()`

ENUNCIADO

Dada una lista de números, verifique si existe al menos un número par dentro de la colección.

DATOS DE ENTRADA	SALIDA ESPERADA
<code>[7, 11, 13, 20, 25]</code>	<code>¿Hay algún par? true (el 20 es par)</code>

#14

Verificar condición en todos los elementos

`allMatch()`

ENUNCIADO

Dada una lista de notas, determine si todas son mayores o iguales a 3.0.

DATOS DE ENTRADA	SALIDA ESPERADA
<code>[4.0, 3.5, 4.2, 5.0, 3.8]</code>	<code>¿Todas >= 3.0? true</code>

#15

Verificar ausencia de un elemento

`noneMatch()`

ENUNCIADO

Dada una lista de usuarios, verifique que ninguno tenga el nombre "root".

DATOS DE ENTRADA	SALIDA ESPERADA
<code>["juan", "maria", "admin", "pedro", "soporte"]</code>	<code>¿Ninguno es "root"? true</code>



RETO FINAL INTEGRADOR

Combina: `filter` · `map` · `sorted` · `peek` · `collect`

ENUNCIADO

Se tiene una lista de estudiantes con nombre y promedio. Construya una solución usando Streams que permita: filtrar aprobados (promedio ≥ 3.0), convertir nombres a mayúsculas, ordenar por promedio de mayor a menor, mostrar cada estudiante procesado en consola, y guardar el resultado en una nueva lista.

DATOS DE ENTRADA

Nombre	Promedio	Estado	
Ana	4.5	✓ Aprobado	
Carlos	3.2	✓ Aprobado	
Pedro	2.8	✗ Reprobado	
Laura	4.8	✓ Aprobado	
Andrés	3.9	✓ Aprobado	
María	2.5	✗ Reprobado	

SALIDA ESPERADA

SALIDA ESPERADA

```
Procesado: LAURA 4.8
Procesado: ANA 4.5
Procesado: ANDRÉS 3.9
Procesado: CARLOS 3.2

Lista final:
[LAURA(4.8), ANA(4.5),
 ANDRÉS(3.9), CARLOS(3.2)]
```